

Chapter 5: Methods

**Starting Out with Java:
From Control Structures through Objects**

Fifth Edition

by Tony Gaddis

Chapter Topics

Chapter 5 discusses the following main topics:

- Introduction to Methods
- Passing Arguments to a Method
- More About Local Variables
- Returning a Value from a Method
- Problem Solving with Methods

Why Write Methods?

- Methods are commonly used to break a problem down into small manageable pieces. This is called *divide and conquer*.
- Methods simplify programs. If a specific task is performed in several places in the program, a method can be written once to perform that task, and then be executed anytime it is needed. This is known as *code reuse*.

void Methods and Value-Returning Methods

- A `void` method is one that simply performs a task and then terminates.

```
System.out.println("Hi!");
```

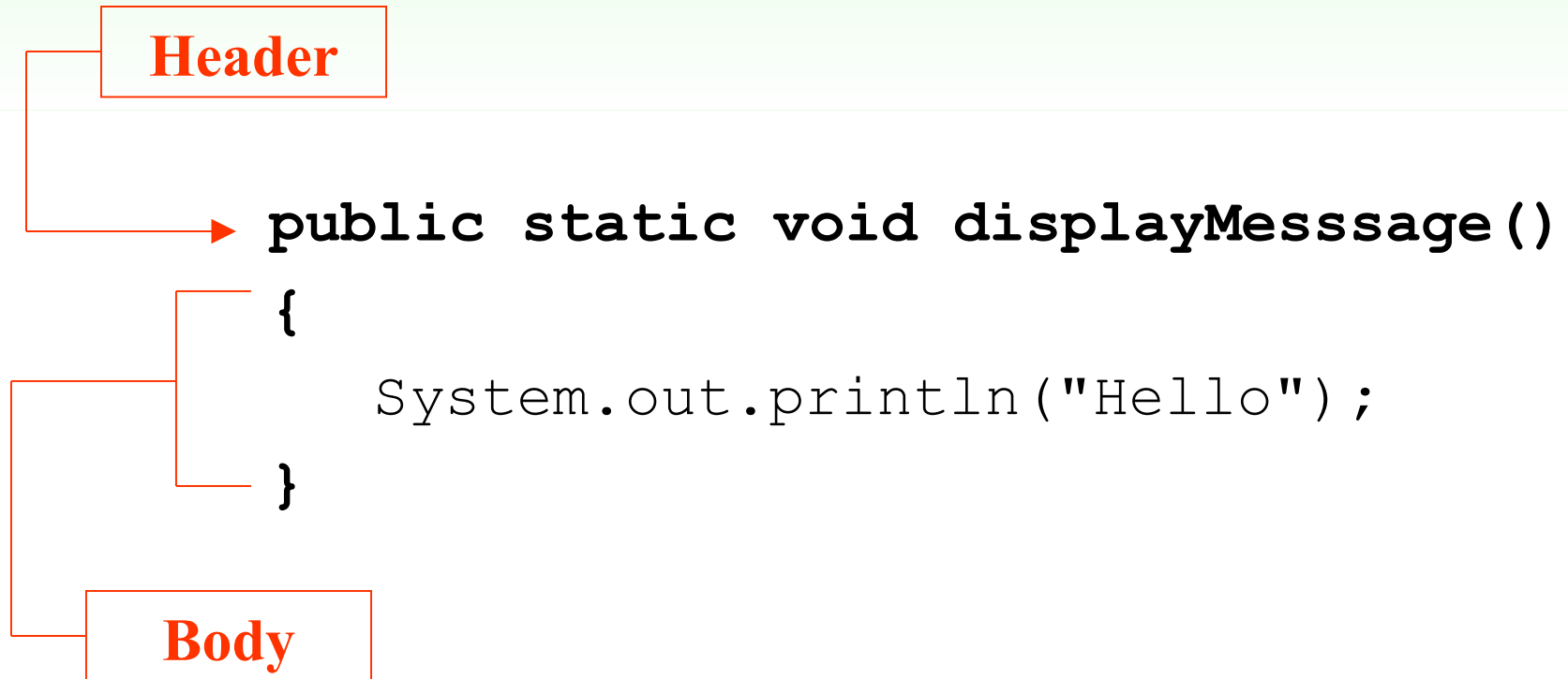
- A value-returning method not only performs a task, but also sends a value back to the code that called it.

```
int number = Integer.parseInt("700");
```

Defining a `void` Method

- To create a method, you must write a definition, which consists of a *header* and a *body*.
- The method header, which appears at the beginning of a method definition, lists several important things about the method, including the method's name.
- The method body is a collection of statements that are performed when the method is executed.

Two Parts of Method Declaration



Parts of a Method Header

Method
Modifiers

Return
Type

Method
Name

Parentheses

The diagram illustrates the components of a Java method header. Four red labels at the top point to specific parts of the code snippet below: 'Method Modifiers' points to 'public static', 'Return Type' points to 'void', 'Method Name' points to 'displayMessage', and 'Parentheses' points to '()'. The code snippet is as follows:

```
public static void displayMessage ()  
{  
    System.out.println("Hello");  
}
```

Parts of a Method Header

- Method modifiers
 - `public`—method is publicly available to code outside the class
 - `static`—method belongs to a class, not a specific object.
- Return type—`void` or the data type from a value-returning method
- Method name—name that is descriptive of what the method does
- Parentheses—contain nothing or a list of one or more variable declarations if the method is capable of receiving arguments.

Calling a Method

- A method executes when it is called.
- The `main` method is automatically called when a program starts, but other methods are executed by method call statements.

`displayMessage()` ;

- Notice that the method modifiers and the `void` return type are not written in the method call statement. Those are only written in the method header.
- Examples: `SimpleMethod.java`, `LoopCall.java`, `CreditCard.java`, `DeepAndDeeper.java`

Documenting Methods

- A method should always be documented by writing comments that appear just before the method's definition.
- The comments should provide a brief explanation of the method's purpose.
- The documentation comments begin with `/*` and end with `*/`.

Passing Arguments to a Method

- Values that are sent into a method are called arguments.

```
System.out.println("Hello");  
number = Integer.parseInt(str);
```

- The data type of an argument in a method call must correspond to the variable declaration in the parentheses of the method declaration. The parameter is the variable that holds the value being passed into a method.
- By using parameter variables in your method declarations, you can design your own methods that accept data this way. See example: PassArg.java

Passing 5 to the `displayValue` Method

`displayValue(5) ;` The argument 5 is copied into the parameter variable **num**.



```
public static void displayValue(int  
    num)  
{  
    System.out.println("The value is " + num) ;  
}
```

The method will display The value is 5

Argument and Parameter Data Type Compatibility

- When you pass an argument to a method, be sure that the argument's data type is compatible with the parameter variable's data type.
- Java will automatically perform widening conversions, but narrowing conversions will cause a compiler error.

```
double d = 1.0;  
displayValue(d);
```

**Error! Can't convert
double to int**

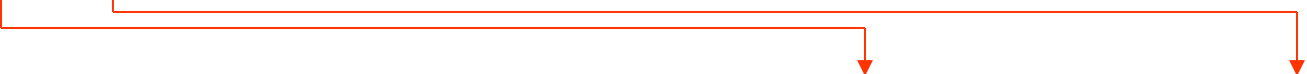


Passing Multiple Arguments

The argument 5 is copied into the **num1** parameter.

The argument 10 is copied into the **num2** parameter.

`showSum(5, 10);` **NOTE: Order matters!**



The diagram consists of a red line that starts below the number 5 in the function call, extends to the right, and then branches into two downward-pointing arrows. The first arrow points to the `double num1` parameter in the method signature, and the second arrow points to the `double num2` parameter.

```
public static void showSum(double num1, double num2)
{
    double sum;    //to hold the sum
    sum = num1 + num2;
    System.out.println("The sum is " + sum);
}
```

Arguments are Passed by Value

- In Java, all arguments of the primitive data types are *passed by value*, which means that only a copy of an argument's value is passed into a parameter variable.
- A method's parameter variables are separate and distinct from the arguments that are listed inside the parentheses of a method call.
- If a parameter variable is changed inside a method, it has no affect on the original argument.
- See example: `PassByValue.java`

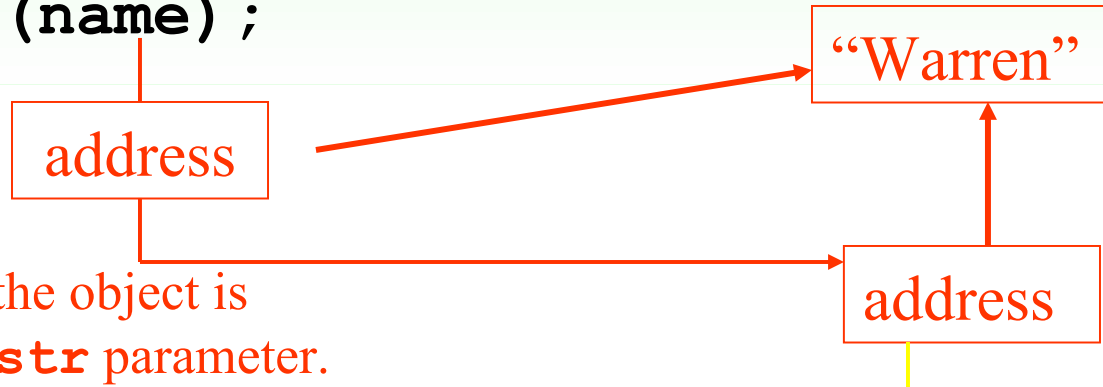
Passing Object References to a Method

- Recall that a class type variable does not hold the actual data item that is associated with it, but holds the memory address of the object. A variable associated with an object is called a reference variable.
- When an object such as a `String` is passed as an argument, it is actually a reference to the object that is passed.

Passing a Reference as an Argument

Both variables reference the same object

`showLength (name) ;`



The address of the object is copied into the **str** parameter.

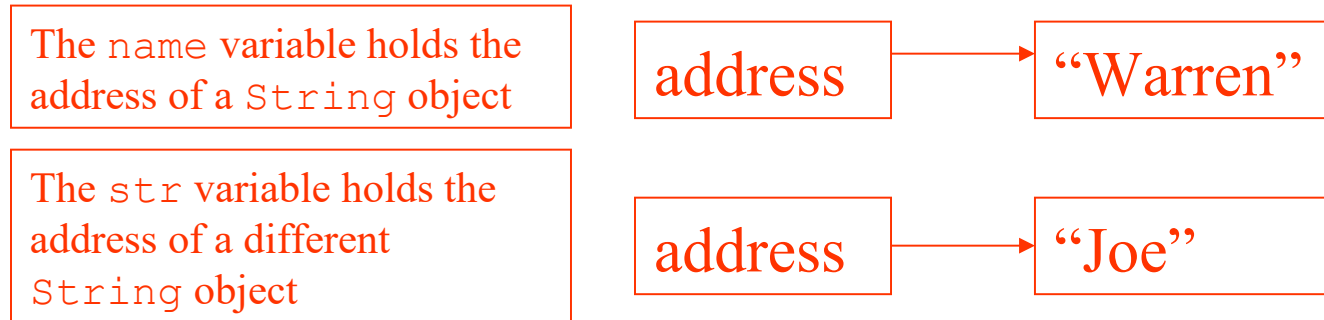
```
public static void showLength(String str)
{
    System.out.println(str + " is " +
        str.length() + " characters
        long.");
    str = "Joe" // see next slide
}
```

Strings are Immutable Objects

- Strings are immutable objects, which means that they cannot be changed. When the line

```
str = "Joe";
```

is executed, it cannot change an immutable object, so creates a new object.



- See example: PassString.java

@param Tag in Documentation Comments

- You can provide a description of each parameter in your documentation comments by using the `@param` tag.
- General format

```
@param parameterName Description
```
- See example: `TwoArgs2.java`
- All `@param` tags in a method's documentation comment must appear after the general description. The description can span several lines.

More About Local Variables

- A local variable is declared inside a method and is not accessible to statements outside the method.
- Different methods can have local variables with the same names because the methods cannot see each other's local variables.
- A method's local variables exist only while the method is executing. When the method ends, the local variables and parameter variables are destroyed and any values stored are lost.
- Local variables are not automatically initialized with a default value and must be given a value before they can be used.
- See example: `LocalVars.java`

Returning a Value from a Method

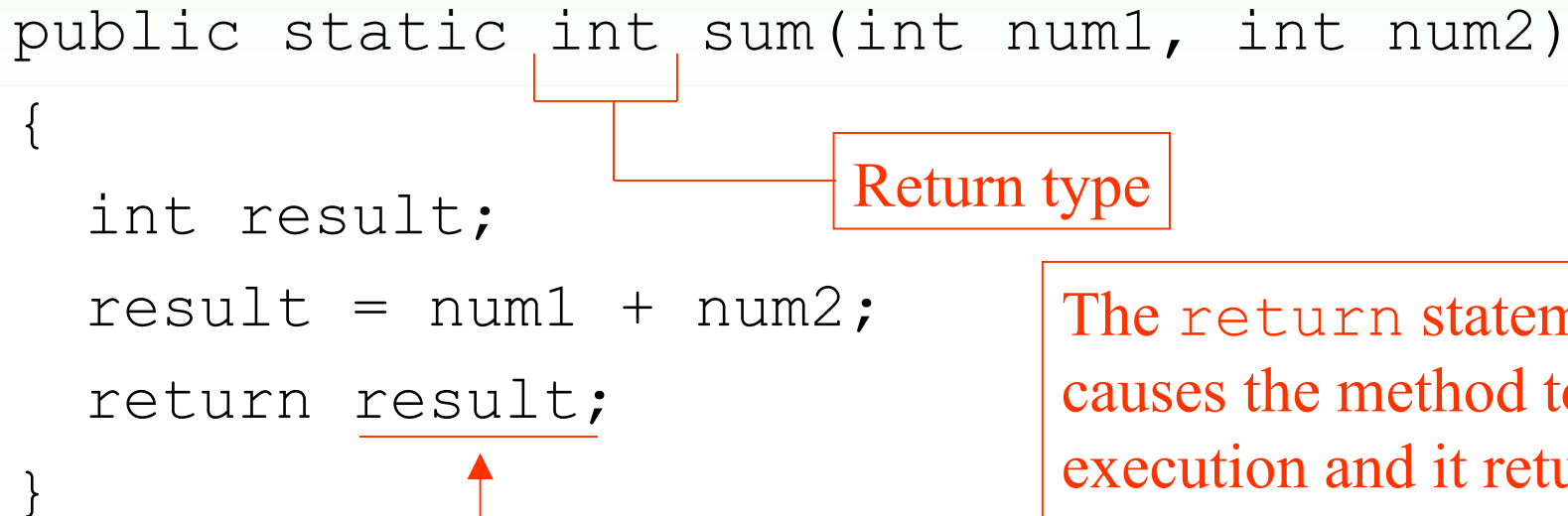
- Data can be passed into a method by way of the parameter variables. Data may also be returned from a method, back to the statement that called it.

```
int num = Integer.parseInt("700");
```

- The string “700” is passed into the `parseInt` method.
- The `int` value 700 is returned from the method and assigned to the `num` variable.

Defining a Value-Returning Method

```
public static int sum(int num1, int num2)
{
    int result;
    result = num1 + num2;
    return result;
}
```



Return type

This expression must be of the same data type as the return type

The return statement causes the method to end execution and it returns a value back to the statement that called the method.

Calling a Value-Returning Method

```
total = sum(value1, value2);
```

20

40

```
public static int sum(int num1, int num2)
```

{

```
    int result;
```

```
    result = num1 + num2;
```

```
    return result;
```

```
}
```

60

@return Tag in Documentation Comments

- You can provide a description of the return value in your documentation comments by using the `@return` tag.

- General format

`@return` Description

- See example: `ValueReturn.java`
- The `@return` tag in a method's documentation comment must appear after the general description. The description can span several lines.

Returning a boolean Value

- Sometimes we need to write methods to test arguments for validity and return true or false

```
public static boolean isValid(int number)
{
    boolean status;
    if(number >= 1 && number <= 100)
        status = true;
    else
        status = false;
    return status;
}
```

Calling code:

```
int value = 20;
If(isValid(value))
    System.out.println("The value is within range");
else
    System.out.println("The value is out of range");
```

Returning a Reference to a String Object

```
customerName = fullName("John", "Martin");
```

```
public static String fullName(String first, String last)
{
    String name;
    name = first + " " + last;
    return name;
}
```

address

"John Martin"

Local variable name holds the reference to the object. The return statement sends a copy of the reference back to the call statement and it is stored in customerName.

See example:
ReturnString.java

Problem Solving with Methods

- A large, complex problem can be solved a piece at a time by methods.
- The process of breaking a problem down into smaller pieces is called *functional decomposition*.
- See example: SalesReport.java
- If a method calls another method that has a `throws` clause in its header, then the calling method should have the same `throws` clause.

Calling Methods that Throw Exceptions

- Note that the `main` and `getTotalSales` methods in *SalesReport.java* have a `throws IOException` clause.
- All methods that use a `Scanner` object to open a file must throw or handle `IOException`.
- You will learn how to handle exceptions in Chapter 12.
- For now, understand that Java required any method that interacts with an external entity, such as the file system to either throw an exception to be handles elsewhere in your application or to handle the exception locally.